5 Queues and Deques

Before you begin: Join our book community on Discord

We have explored the inner workings of **stacks**, a data structure governed by the *LIFO* (Last in First out) principle. Now, let's turn our attention to **queues**, a similar yet distinct data structure. While stacks prioritize the most recent additions, queues operate on a *FIFO* (First in First out) basis, prioritizing the earliest entries. We will delve into the mechanics of queues and then explore **deques**, a versatile hybrid data structure that combines elements of both stacks and queues. By the end of this chapter, you will have a solid understanding of these fundamental data structures and their practical applications.

In this chapter, we will cover the following topics:

- The queue data structure
- The deque data structure
- Adding elements to a queue and a deque
- Removing elements from a queue and a deque
- Simulating circular queues with the *Hot Potato* game
- Checking whether a phrase is a palindrome with a deque
- Different problems we can resolve using queues and deques

The queue data structure

Queues are all around us in everyday life. Think of the line to buy a movie ticket, the cafeteria queue at lunchtime, or the checkout line at the grocery store. In each of these scenarios, the underlying principle is the same: the first person to join the queue is the first one to be served.

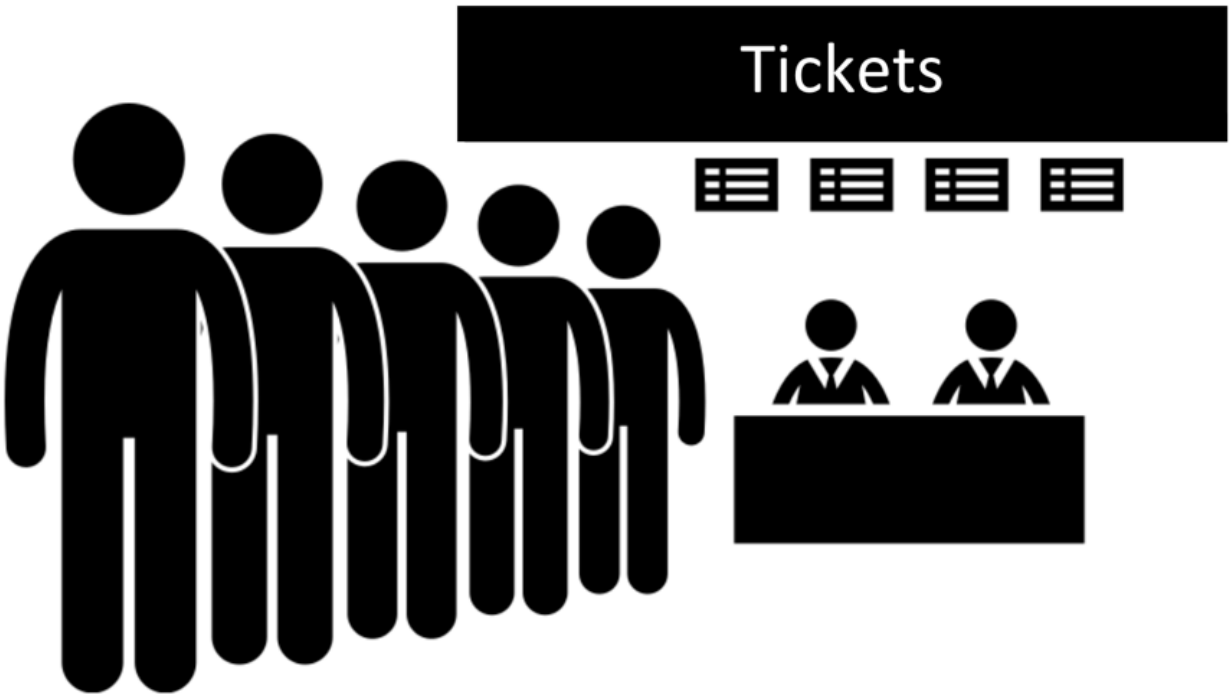

Figure 5.1: Real life queue example: a group of people standing in line to buy a ticket

An extremely popular example in computer science is the printing line. Let's say we need to print five documents. We open each document and click on the *Print* button. Each document will be sent to the print line. The first document that we asked to be printed is going to be printed first and so on, until all the documents are printed.

In the realm of data structures, a queue is a linear collection of elements that adheres to the *First In, First Out* (FIFO) principle, often referred to as *First Come, First Served*. New elements are always added at the rear (end) of the queue, while removal of elements always occurs at the front (beginning).

Let's put these concepts into practice by creating our own Queue class using JavaScript and TypeScript.

Creating the Queue class

We are going to create our own class to represent a queue. The source code for this chapter is available inside the src/05-queue-deque folder on GitHub. We will start by creating the queue.js file which will contain our class that represents a stack using an array-based approach.

In this book, we will take an incremental approach to building our understanding of data structures. We will leverage the concepts we mastered in the previous chapter and gradually increase the complexity as we progress (so make sure you do not skip chapters).

This approach will allow us to build a solid foundation and tackle more intricate structures with confidence.

First, we will declare our Queue class:

```
class Queue {

 #items = [];

 // other methods

}
```

We need a data structure that will store the elements of the queue. We can use an array to do this as we are already familiar with the array data structure, and we have also learned in the previous chapter that an array-based approach is preferred compared to an object-based approach.

And again, we will prefix the variable items with the hash # prefix to indicate this property is private and can only be referenced inside the Queue class, hence, allowing us to protect the data and follow the FIFO principle when it comes to the insertion and removal of elements.

The following methods will be available in the Queue class:

- enqueue(item): This method adds a new item at the end of the queue.
- dequeue(): This method removes the first item from the beginning of the queue. It also returns the removed item.
- front(): This method returns the first element from the beginning of the queue. The queue is not modified (it does not remove the element; it only returns the element for information purposes). This is like the peek method from the Stack class.
- isEmpty(): This method returns true if the queue does not contain any elements, and false if the size of the stack is bigger than 0.
- clear(): This method removes all the elements of the queue.
- size(): This method returns the number of elements that the queue contains.

We will code each method in the following subsections.

Enqueueing elements to end of the queue

The first method that we will implement is the enqueue method. This method is responsible for adding new elements to the queue, with one especially important detail: we can only add new items at the rear of the queue as follows:

```
enqueue(item) {
  this.#items.push(item);
}
```

As we are using an array to store the elements of the queue, we can use the push method from the JavaScript Array class that will add a new item to the end of the array.
The enqueue method has the same implementation as the push method from the Stack class; from a code standpoint, we are only changing the name of the method.

Dequeuing elements from beginning of the queue

Next, we are going to implement the dequeue method. This method is responsible for removing the items from the queue. As the queue uses the FIFO principle, the item at the beginning of the queue (index 0 of our internal array) is removed. For this reason, we can use the shift method from the JavaScript Array class as follows:

```
dequeue() {
  return this.#items.shift();
}
```

The shift method from the Array class removes the first element from an array and returns it. If the array is empty, undefined is returned and the array is not modified. So this works perfectly with the behavior of the queue.

The enqueue method has constant time complexity (*O(1)*). The dequeue method can have linear time complexity (*O(n)*), since we are using the shift method to remove the first element of an array, it can lead to *O(n)* performance in the worst case as the remaining elements need to be shifted down.

Peeking the element from the front of the queue

Next, we will implement additional helper methods in our class. If we would like to know what the front element of the queue is, we can use the front method. This method will return the item that is located at the index 0 of our internal array:

```
front() {
  return this.#items[0];
```

```
}
```

Verifying if it is empty, the size and clearing the queue

The next methods we will create are the isEmpty method, size getter, and the clear method. These three methods have the exact same implementation as the Stack class:

```
 isEmpty() {
  return this.#items.length === 0;
 }
 get size() {
  return this.#items.length;
 }
 clear() {
  this.#items = [];
 }
```

Using the isEmpty method, we can simply verify whether the length of the internal array is 0.

For the size, we create a getter for the size of the queue, which is simply the length of the internal array.

And for the clear method, we can simply assign a new empty array that will represent an empty queue.

Finally, we have the toString method, with the same code as the Stack class as follows:

```
toString() {
 if (this.isEmpty()) {
  return 'Empty Queue';
 } else {
  return this.#items.map(item => {
   if (typeof item === 'object' && item !== null) {
    return JSON.stringify(item);
```

```
    } else {
      return item.toString();
    }
  }).join(', ');
 }
}
```

Exporting the Queue data structure as a library class

We have created a file src/05-queue-deque/queue.js with our Queue class. And we would like to use the Queue class in a different file for testing for easy maintainability of our code (src/05-queue-deque/01-using-queue-class.js). How can we achieve this?

We covered this in the previous chapter as well. We will use the module.exports from CommonJS Module to expose our class:

```
// queue.js
class Queue {
  // our Queue class implementation
}
module.exports = Queue;
```

Using the Queue class

Now it is time to test and use our Queue class! As discussed in the previous subsection, let's go ahead and create a separate file so we can write as many tests as we like: src/05-queue-deque/01-using-queue-class.js.

The first thing we need to do is to import the code from the queue.js file and instantiate the Queue class we just created:

```
const Queue = require('./queue');
const queue = new Queue();
```

Next, we can verify whether it is empty (the output is true, because we have not added any elements to our queue yet):

```
  console.log(queue.isEmpty()); // true
```

Next, let's simulate a printer's queue. Suppose we have three documents open in a computer. And we click on the print button in each document. By doing so, it will enqueue the documents to the queue in the order the print button was clicked:

```
queue.enqueue({ document: 'Chapter05.docx', pages: 20 });

queue.enqueue({ document: 'JavaScript.pdf', pages: 60 });

queue.enqueue({ document: 'TypeScript.pdf', pages: 80 });
```

If we call the front method, it is going to return the file Chapter05.docx, because it was the first document that was added to the queue to be printed:

```
console.log(queue.front()); // { document: 'Chapter05.docx', pages: 20 }
```

Let's also check the queue size:

```
  console.log(queue.size); // 3
```

Now let's "print" all documents in the queue by dequeuing them until the queue is empty:

```
// print all documents

while (!queue.isEmpty()) {

 console.log(queue.dequeue());

}
```

The following diagram shows the dequeue operation when printing the first document from the queue:

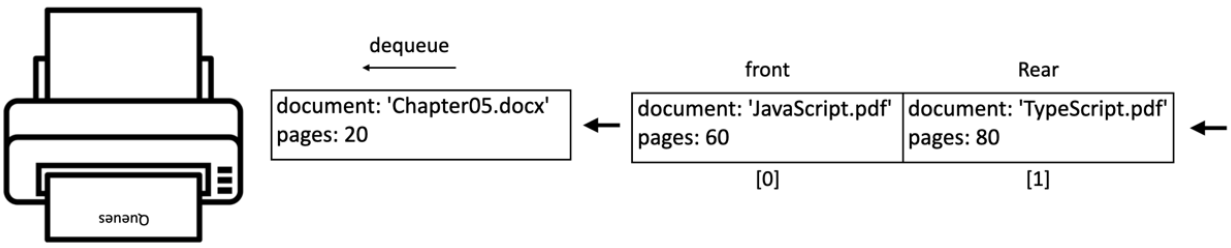

Figure 5.2: Simulation of a printer queue

Reviewing the efficiency of our Queue class

Let's review the efficiency of each method of our queue class by considering the Big O notation in terms of time of execution:

Method	Complexity	Explanation
enqueue	*O(1)*	Adding an element to the end of an array is usually a constant-time operation.
dequeue	*O(n)*	Removing the first element requires shifting all remaining elements, taking time proportional to the queue's size.
front	*O(1)*	Directly accessing the first element by index is a constant-time operation.
isEmpty	*O(1)*	Checking the length property of an array is a constant-time operation.
size	*O(1)*	Accessing the length property is a constant-time operation.
clear	*O(1)*	Overwriting the internal array with an empty array is considered constant time.
toString	*O(n)*	Iterating over the elements, potentially stringifying them, and joining them into a string takes time proportional to the number of elements.

Table 5.1:

The dequeue operation is generally the most performance-sensitive operation for a queue implemented using an array. This is due to the need to shift elements after removing the first one. There are alternative queue implementations (for example using a linked list, which we will cover in the next chapter, or a **circular queue**) that can optimize the dequeue operation to have constant time complexity in most cases. We will create a circular queue later in this chapter.

The deque data structure

The **deque** data structure, also known as the **double-ended queue**, is a special queue that allows us to insert and remove elements from the end or from the front of the deque.

A deque can be used to store a user's web browsing history. When a user visits a new page, it is added to the front of the deque. When the user navigates back, the most recent page is removed from the front, and when the user navigates forward, a page is added back to the front.

Another application would be an undo/redo feature. We learned we can use two stacks for this feature in the last chapter, but we can also use a deque as an alternative. User actions are pushed onto the deque, and undo operations pop actions off the front, while redo operations push them back on.

Creating the Deque class

As usual, we will start by declaring the Deque class located in the file src/05-queue-deque/deque.js:

```
class Deque {

  #items = [];

}
```

We will continue using an array-based implementation for our data structure. And given the deque data structure allows inserting and removing from both ends, we will have the following methods:

- addFront(item): This method adds a new element at the front of the deque.
- addRear(item): This method adds a new element at the back of the deque.
- removeFront(): This method removes the first element from the deque.
- removeRear(): This method removes the last element from the deque.
- peekFront(): This method returns the first element from the deque.
- peekRear(): This method returns the last element from the deque.

The Deque class also implements the isEmpty, clear, size, and toString methods (you can check the complete source code by downloading the source code bundle for this book). The code for these is the same as for the Queue class.

Adding elements to the deque

Let's check both methods that will allow us to add elements to the front and to the back of the deque:

```
addFront(item) {
  this.#items.unshift(item);
}
addRear(item) {
  this.#items.push(item);
}
```

The addFront method inserts an element at index 0 of the internal array by using the Array.unshift method.

The addRear method inserts an element at the end of the deque. It has the same implementation as the enqueue method from the Queue class.

Removing elements from the deque

The methods to remove from both the front and the back of the deque are presented as follows:

```
removeFront() {
  return this.#items.shift();
}
removeRear() {
  return this.#items.pop();
}
```

The removeFront method removes and returns the element at the beginning (front) of the deque. If the deque is empty, it returns undefined. It has the same implementation as the dequeue method from the Queue class.

The removeRear method removes and returns the element at the end (rear) of the deque. If the deque is empty, it returns undefined. It has the same implementation as the pop method from the Stack class.

Peeking elements of the deque

Finally, let's check the peek methods as follows:

```
peekFront() {
  return this.#items[0];
}
peekRear() {
  return this.#items[this.#items.length - 1];
}
```

The peekFront method allows you to look at (peek) the element at the beginning (front) of the deque without removing it. It has the same implementation as the peek method from the Queue class.

The peekRear method allows you to look at (peek) the element at the end (rear) of the deque without removing it. It has the same implementation as the peek method from the Stack class.

Note the similarities of the implementation of the deque methods with the Stack and Queue classes. We can say the deque data structure is a hybrid version of the stack and queue data structures. Please refer to the Queue and Stack efficiency review to check the time complexity for these methods.

Using the Deque class

It is time to test our Deque class (src/05-queue-deque/03-using-deque-class.js). We will use it in the scenario of a browser's "*Back*" and "*Forward*" button functionality. Let's see how this could be implemented using our Deque class:

```
const Deque = require('./deque');
class BrowserHistory {
  #history = new Deque(); // {1}
  #currentPage = null; // {2}
  visit(url) {
    this.#history.addFront(url); // {3}
    this.#currentPage = url; // {4}
  }
```

```
 goBack() {
  if (this.#history.size() > 1) { // {5}
    this.#history.removeFront();  // {6}
    this.#currentPage = this.#history.peekFront(); // {7}
  }
 }
 goForward() {
  if (this.#currentPage !== this.#history.peekBack()) { // {8}
    this.#history.addFront(this.#currentPage);  // {9}
    this.#currentPage = this.#history.removeFront(); // {10}
  }
 }
 get currentPage() { // returns the current page for information
  return this.#currentPage;
 }
}
```

Following is the explanation:

- A Deque named history is created to store URLs of visited pages ({1}).
  The currentPage variable keeps track of the currently displayed page ({2}).
- The visit(url) method adds the new url to the front of the history deque ({3}) and updates the currentPage to the new URL ({4}).
- The goBack() method:
  - Checks if there are at least two pages in the history (current and previous - {5}).
  - If so, it removes the current page from the front of the history deque ({6}).
  - Updates currentPage to the now-front element, which represents the previous page ({7}).

- The goForward() method:
    - Checks if the currentPage is different from the last element in the history deque (meaning there is a "next" page - {8}).
    - If so, adds the current page back to the front of the history deque ({9}).
    - Removes and sets currentPage to the now-front element, which was the "next" page ({10}).

With our browser simulation ready, we can use it:

```
const browser = new BrowserHistory();

browser.visit('loiane.com');

browser.visit('https://loiane.com/about'); // click on About menu

browser.goBack();

console.log(browser.currentPage); // loiane.com

browser.goForward();

console.log(browser.currentPage); // https://loiane.com/about
```

We will simulate visiting the website **https://loiane.com**, which is the author's blog. Next, we will visit the About page so we can add another URL to the browser's history. Then, we can click on the "Back" button to go back to the home page. And we can also click on the "Next" button to go back the About page. And of course, we can peek what is the current page or the history as well. The following image exemplifies this simulation:

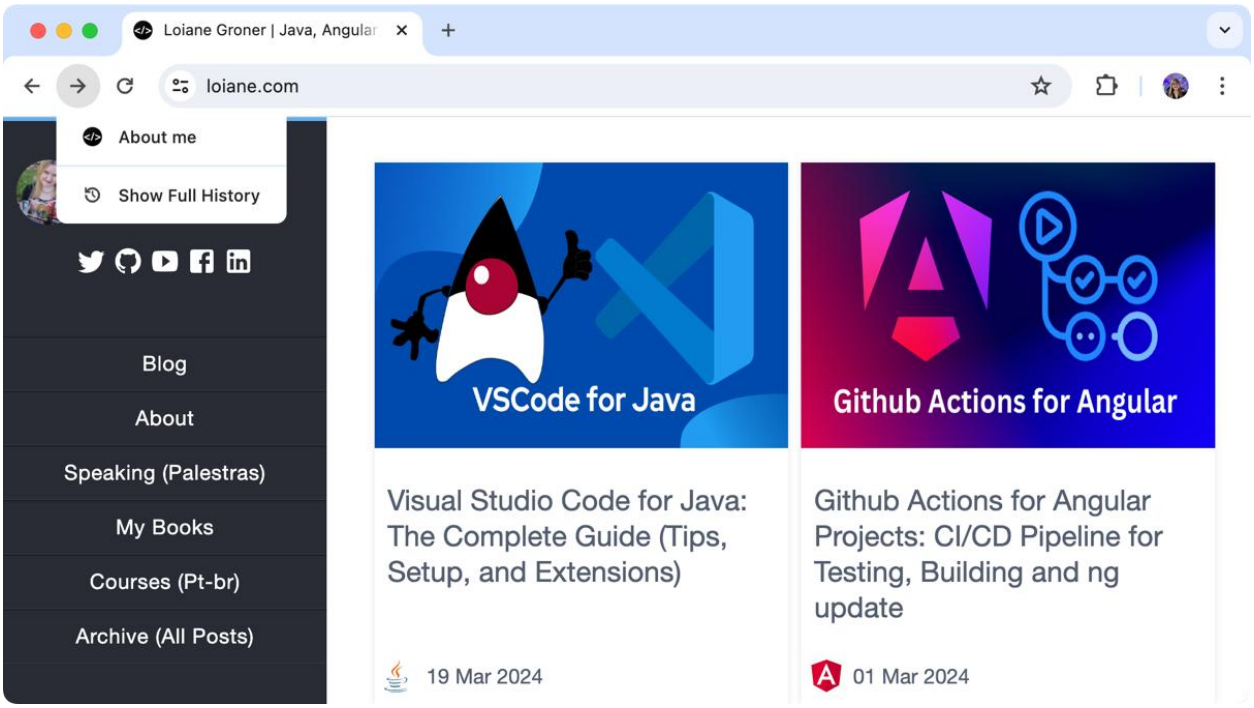

Figure 5.3: Simulation of a browser Back and Next buttons

Creating the Queue and Deque classes in TypeScript

With the JavaScript implementation done, we can rewrite our Queue and Deque classes using TypeScript:

```
// src/05-queue-deque/queue.ts

class Queue<T> {

  private items: T[] = [];

  enqueue(item: T): void {}

  // all other methods are the same as in JavaScript

}

export default Queue;
```

And the Deque class:

```
// src/05-queue-deque/deque.ts

class Deque<T> {

  private items: T[] = [];

  addFront(item: T): void {}
```

```
  addRear(item: T): void {}

  // all other methods are the same as in JavaScript

}

export default Deque;
```

We will use the generics to make our data structures flexible and have items of only one type. The code inside the methods is the same as the JavaScript implementation.

Solving problems using queues and deques

Now that we know how to use the Queue and Deque classes, let's use them to solve some computer science problems. In this section, we will cover a simulation of the *Hot Potato* game with queues and how to check whether a phrase is a *palindrome* with deques.

The circular queue: the Hot Potato game

The Hot Potato game is a classic children's game where participants form a circle and pass around an object (the "hot potato") as fast as they can while music plays. When the music stops, the person holding the potato is eliminated. The game continues until only one person remains.

The CircularQueue class

We can perfectly simulate this game using a **Circular Queue** implementation:

```
class CircularQueue {

  #items = [];

  #capacity = 0; // {1}

  #front = 0; // {2}

  #rear = -1; // {3}

  #size = 0; // {4}

  constructor(capacity) { // {5}

    this.#items = new Array(capacity);

    this.#capacity = capacity;

  }

  get size() { return this.#size; }
```

}

A circular queue is a queue implemented using a fixed-size array, meaning a pre-defined capacity ({1}), where the front ({2}) and rear ({3}) pointers can "wrap around" to the beginning of the array when they reach the end. This efficiently reuses the space in the array, avoiding unnecessary resizing. The front pointer is initialized to 0, pointing to the first element's position. The rear pointer is initialized to -1, indicating an empty queue.
The size property ({4}) tracks the current number of elements in the queue.

When we create a circular queue, we need to inform how many elements we are planning to store ({5}).

Let's review the enqueue and isFull methods next:

```
enqueue(item) {
  if (this.isFull()) { // {6}
    throw new Error("Queue is full");
  }
  this.#rear = (this.#rear + 1) % this.#capacity; // {7}
  this.#items[this.#rear] = item; // {8}
  this.#size++; // {9}
}
isFull() { return this.#size === this.#capacity; }
```

Before we add any elements to queue, we need to check if it is not full, meaning the size is the same as the capacity ({6}). With a fixed capacity, this makes the circular queue predictable in terms of memory usage, but this can also be viewed as a limitation.

If the queue is not full, we increment the rear pointer by 1. The key point here is to use the modulo operator (%) to wrap rear back to 0 if it reaches the end of the array ({7}). Then we insert the item at the new rear position ({8}) and increment the size counter ({9}).

Finally, we have the dequeue and isEmpty methods:

```
dequeue() {
  if (this.isEmpty()) { throw new Error("Queue is empty"); } // {10}
  const item = this.#items[this.#front]; // {11}
```

```
  this.#size--; // {12}
  if (this.isEmpty()) {
    this.#front = 0; // {13}
    this.#rear = -1; // {14}
  } else {
    this.#front = (this.#front + 1) % this.#capacity; // {15}
  }
  return item; // {16}
}
isEmpty() { return this.#size === 0;  }
```

To remove the item at the front of the queue, first, we need to check the queue size ({10}). If the queue is not empty, we can retrieve the item that is currently stored at the front position ({11}) so we can return it later ({16}). Then, we decrement the size counter ({12}).

If the queue is not empty after dequeuing, we need to increment the front pointer by 1 and wrap around it using the modulo operator ({15}). If the queue is empty after dequeuing, we reset both front ({13}) and rear ({14}) pointers to their initial values.

The biggest advantage of the circular queue is both enqueueing and dequeuing operations are generally *O(1)* (constant time) due to direct manipulation of pointers.

The Hot Potato game simulation

With the new class ready to be used, let's put it in practice the Hot Potato game simulation:

```
function hotPotato(players, numPasses) {
  const queue = new CircularQueue(players.length); // {1}
  for (const player of players) { // {2}
    queue.enqueue(player);
  }
  while (queue.size > 1) { // {3}
    for (let i = 0; i < numPasses; i++) { // {4}
```

```
    queue.enqueue(queue.dequeue()); // {5}
  }
  console.log(`${queue.dequeue()} is eliminated!`); // {6}
 }
 return queue.dequeue(); // {7} The winner
}
```

This function takes an array of players and a number num representing the number of times the "potato" is passed before a player is eliminated and it returns the winner.

First, we create a circular queue ({1}) with the numbers of players and we enqueue all players ({2}).

We run a loop until only one player remains ({3}):

- Another loop will pass the potato numPasses times ({4}) by dequeueing and then re-enqueueing the same player ({5}).
- The player at the front is removed and announced as eliminated ({6}).

The last remaining player is dequeued and declared the winner ({7}).

We can use the following code to try the hotPotato algorithm:

```
const players = ["Violet", "Feyre", "Poppy", "Oraya", "Aelin"];
const winner = hotPotato(players, 7);
console.log(`The winner is: ${winner}!`);
```

The execution of the algorithm will have the following output:

```
Poppy is eliminated!
Feyre is eliminated!
Aelin is eliminated!
Oraya is eliminated!
The winner is: Violet!
```

This output is simulated in the following diagram:

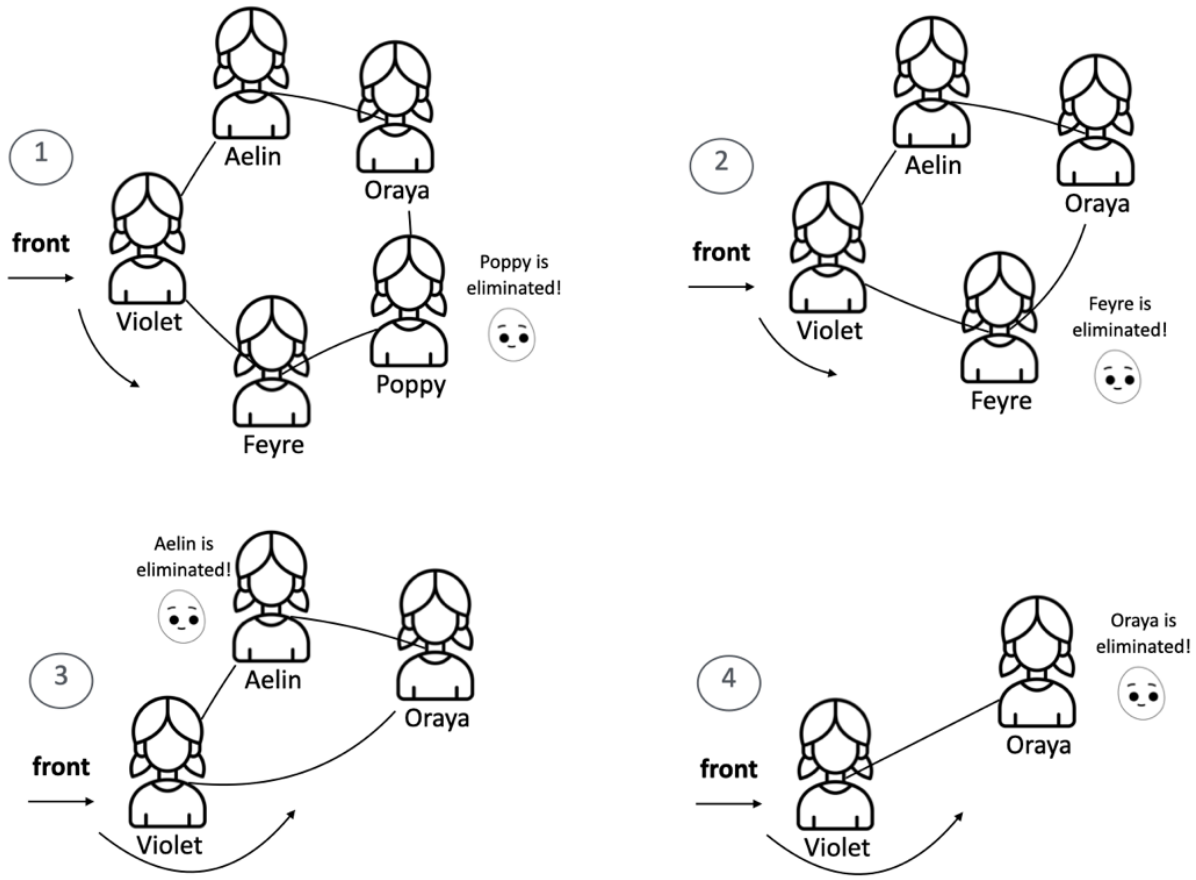

Figure 5.4: The Hot Potato game simulation using circular queue

You can change the number of passes using the hotPotato function to simulate different scenarios.

Palindrome checker

The following is the definition of a **palindrome** according to Wikipedia:

A palindrome is a word, phrase, number, or other sequence of characters which reads the same backward as forward, such as madam or racecar.

There are different algorithms we can use to verify whether a phrase or string is a palindrome. The easiest way is reversing the string and comparing it with the original string. If both strings are equal, then we have a palindrome. We can also use a stack to do this, but the easiest way of solving this problem using a data structure is using a deque: characters are added to the deque, and then removed from both ends simultaneously. If the removed characters match throughout the process, the string is a palindrome.

The following algorithm uses a deque to solve this problem:

```
const Deque = require('./deque');

function isPalindrome(word) {
 if (word === undefined || word === null ||
   (typeof word === 'string' && word.length === 0)) { // {1}
  return false;
 }
 const deque = new Deque(); // {2}
 word = word.toLowerCase().replace(/\s/g, ''); // {3}
 for (let i = 0; i < word.length; i++) {
  deque.addRear(word[i]); // {4}
 }
 while (deque.size() > 1) { // {5}
  if (deque.removeFront() !== deque.removeRear()) { // {6}
    return false;
  }
 }
 return true;
}
```

Before we start with the algorithm logic, we need to verify whether the string that was passed as a parameter is valid with the edge cases ({1}). If it is not valid, then we return false because an empty string or non-existent word cannot be considered a palindrome.

We will use the Deque class we implemented in this chapter ({2}). As we can receive a string with both lowercase and capital letters, we will transform all letters to lowercase, and we will also remove all the spaces ({3}). If you want to, you can also remove all special characters such as !?-() and so on. To keep this algorithm simple, we will skip this part. Next, we will enqueue all characters of the string to the deque ({4}).

While we have at least two elements in the deque ({5} - if only one character is left, it is a palindrome), we will remove one element from the front and one from the back ({6}). To be a palindrome, both characters removed from the deque need to match. If the characters do not match, then the string is not a palindrome ({7}).

We can use the following code to try the isPalindrome algorithm:

console.log(isPalindrome("racecar")); // Output: true

Exercises

We will resolve an exercise from **LeetCode** using the concepts we learned in this chapter.

Number of Students Unable to Eat Lunch

The exercise we will resolve is the *1700. Number of Students Unable to Eat Lunch* problem available at **https://leetcode.com/problems/number-of-students-unable-to-eat-lunch/**.

When resolving the problem using JavaScript or TypeScript, we will need to add our logic inside the function function countStudents(students: number[], sandwiches: number[]): number {}, which receives a queue of students that would prefer to eat a sandwich 0 or 1 and a stack of sandwiches, which will have the same size.

This is a simulation exercise, according to the problem's description:

- If the student at the front of the queue prefers the sandwich on the top of the stack, they will take it and leave the queue.
- Otherwise, they will leave it and go to the queue's end.
- This continues until none of the queue students want to take the top sandwich and are thus unable to eat.

The key to resolve this problem is to also treat the stack of sandwiches as a queue (*FIFO*) instead of a stack (*LIFO*).

Let's write the countStudents function:

```
function countStudents(students: number[], sandwiches: number[]) {
  while (students.length > 0) { // {1}
    if (students[0] === sandwiches[0]) { // {2}
      students.shift(); // {3}
```

```
        sandwiches.shift(); // {4}
    } else {
        if (students.includes(sandwiches[0])) { // {5}
            let num = students.shift(); // {6}
            students.push(num); // {7}
        } else {
            break; // {8}
        }
    }
 }
 return students.length;
}
```

The while loop keeps running as long as there are students in the queue ({1}).

If the first student's preference(students[0]) matches ({2}) the sandwich at the front (top sandwich sandwiches[0]), both the student ({3}) and sandwich ({4}) are removed from the front of their respective queues.

If there is not a match, we check for potential match using the includes method from the JavaScript Array class to see if any student in the queue would take the current top sandwich ({5}). If there is a potential match, the current student ({6}) is moved to the back of the queue ({7}).

If there is no one left in line who wants the current sandwich, the loop breaks ({8}). This indicates that the remaining students will not be able to eat.

The function returns the length of the students array. This length represents the number of students still in line who could not get a sandwich they liked.

This solution passes all the tests and resolves the problem. The includes check ({5}) is important for efficiency. Without it, the code would unnecessarily rotate the queue even when no student would want the top sandwich, so we get bonus points, although this method is not native for a queue data structure.

The time and space complexity of this function is *O(n^2)*, where *n* is the number of students. This is because the outer loop runs until there are no students left, which in the worst case is *n* iterations.

Inside the loop, there are two operations that can be costly: students.shift(), which is *O(n)*, and students.includes(sandwiches[0]), which is also *O(n)*. Since these operations are nested inside the loop, the total complexity is *O(n^2)*.

The space complexity is *O(1)*, not counting the space needed for the input arrays. This is because the function only uses a fixed amount of additional space to store the num variable.

Can you think of a more optimized approach to resolve this problem that might not involve the queue data structure? During technical interviews, it is important to also think about optimizations. Give it a try, and you will find the solution along with the explanation in the source code, along with the 2034. Time Needed to Buy Tickets problem resolution as well.

Summary

In this chapter, we delved into the fundamental concept of queues and their versatile cousin, the deque. We crafted our own queue algorithm, mastering the art of adding (enqueue) and removing (dequeue) elements in a first-in, first-out (FIFO) manner. Exploring the deque, we discovered its flexibility in inserting and deleting elements from both ends, expanding the possibilities for creative solutions.

To solidify our understanding, we applied the knowledge to real-world scenarios. We simulated the classic Hot Potato game, leveraging a circular queue to model its cyclical nature. Additionally, we created a palindrome checker, demonstrating the deque's power in handling data from both directions. We also tackled a simulation challenge from LeetCode, reinforcing the practical applications of queues in problem-solving.

Now, with a firm grasp of these linear data structures (arrays, stacks, queues and deques), we are ready to venture into the dynamic world of linked lists in the next chapter, where we will unlock even greater potential for complex data manipulation and management.